SK네트웍스 Family AI 과정 10기

데이터 전처리 학습된 인공지능 모델

산출물 단계	데이터 전처리
평가 산출물	학습된 인공지능 모델
제출 일자	2025.06.13
깃허브 경로	https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN10-FINAL-3Team.git
작성 팀원	배민경

1. 모델 목적

- **판례 본문**을 입력으로 받아, 해당 사건의 판결 결과(예: 인용, 기각, 파기환송 등)를 자동 분류하는 모델 구축
- 향후 법률 도우미 LLM의 **판례 요약, 유사 판례 추천** 등에 활용될 수 있음

2. 모델 아키텍처 설계

● 선정 모델

- bert-base-multilingual-cased 기반 **문장 분류 모델 (Sequence Classification)**
- 사전학습 모델을 fine-tuning 하여 판례 데이터에 특화된 분류기로 재학습함

● 아키텍처 개요

계층	**구성 요소**	**역할**
입력층	**Tokenizer + BERT Embedding**	판례 전문 텍스트를 BERT 입력용 토큰 벡터로 변환
인코더	**Transformer Encoder Blocks (12-layer)**	문장 내 단어 간 관계 및 의미 표현 학습
출력층	**Linear(Dense) Layer + Softmax**	8개 판결 결과 중 하나를 예측 (다중 클래스 분류)

● 아키텍처 시각화

[Input 판례 텍스트]

↓

[BERT Tokenizer]

↓

[BERT Encoder (12-layer)]

↓

[CLS 토큰 출력 → Linear Layer]

↓

[Softmax] → [예측 클래스: 기각 / 인용 / 파기환송 ...]

● 설계 근거

- BERT 기반 모델은 긴 문장과 복잡한 문맥을 이해하는 데 매우 적합
- 판례 전문은 문장이 길고 형식적이며, 특정 단어보다 문맥 전체의 의미 흐름이 중요한 경우가 많음
- bert-base-multilingual-cased는 한글 법률 데이터에도 잘 작동하며, 다국어 확장 가능성이 있음
- 사전학습된 모델 구조를 그대로 유지한 채, 출력층만 변경하여 학습 시간 및 자원 효율적

3. 모델 학습 요약

● 데이터 구성

구분	**데이터 수 (건)**
학습 데이터	2126건
검증 데이터	112건

※ 데이터셋은 Stratified Split(라벨 비율 유지) 방식으로 분할되었으며, 테스트셋에서는 소수 클래스(3, 6번) 제거됨

● 성능 평가 결과 (검증셋 기준)

모델	**Accuracy**	**Precision**	**Recall**	**F1 Score**
BERT	0.976	0.98	0.98	0.978

※ 평가지표는 macro 평균 기준으로 산출됨

※ 모델은 전체 클래스에 대해 고르게 예측 성능을 보임

● 일반화 성능 평가

- **테스트셋(미검증 데이터)**에 대해서도 유사한 성능 유지
- **과적합 방지 기법 적용**:
  - Dropout (0.1, BERT 기본값)
  - EarlyStoppingCallback (patience=2, validation loss 기준)
- 학습 정확도와 검증/테스트 정확도 차이가 크지 않아 **과소적합 징후 없음**

4. 저장 및 배포

● 저장 형식

항목	**설명**
저장 파일명	bert_case_classifier_final.pt
저장 형식	PyTorch .pt (state_dict 기반 저장 형식)
저장 방법	torch.save(model.state_dict(), 'bert_case_classifier_final.pt')
불러오기 예시	model.load_state_dict(torch.load('bert_case_classifier_final.pt'))

● 모델 사양 요구 사항

항목	**내용**
프레임워크	**PyTorch 2.0 이상**

실행 환경	GPU 사용 시 학습/추론 속도 향상, CPU 추론 가능
종속 패키지 관리	requirements.txt를 통해 환경 일관성 유지

requirements.txt

```
torch>=2.0.0
transformers>=4.30.0
datasets>=2.10.0
scikit-learn>=1.2.0
pandas>=1.5.0
tqdm
```

● **모델 테스트**

- **최종 저장된 모델(bert_case_classifier_final.pt)을 적재한 후, 임의의 판례 문장을 입력으로 하여 분류 결과를 확인함.**
- **모델은 법률 문장을 입력 받아 미리 정의된 클래스(예: 인용, 기각 등) 중 하나를 예측함.**

```python
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# 모델 및 토크나이저 로드
model_name = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=5)  # 클래스 수에 맞게 조정
model.load_state_dict(torch.load("bert_case_classifier_final.pt"))
model.eval()

# 테스트 문장
text = "피고는 부동산중개업자로서 임대차 계약 시 그릇된 정보를 제공하였다."

# 전처리 및 추론
inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=512)
with torch.no_grad():
    outputs = model(**inputs)
predicted_class = torch.argmax(outputs.logits, dim=1).item()

print(f"예측된 클래스: {predicted_class}")
# 예측된 클래스 출력
```

5. 종합 평가 및 활용 방안

● 모델 안정성

- 동일한 학습 파라미터 및 seed (random_state=42) 하에 학습을 반복할 경우
  모델 저장 및 재로드 후에도 ±0.1% 이내의 정확도 재현 가능

- Hugging Face 기반 Trainer와 PyTorch state_dict 저장 방식을 병행하여
  실험 결과의 일관성 및 재현성 확보

● 일반화 가능성

- 테스트셋(미검증 데이터)에 대해서도 검증셋과 유사한 성능 유지

- 데이터 분할 시 Stratified Split (라벨 분포 유지) 방식 적용 → 클래스 불균형에 대한 과도한 편향 방지

- 다양한 모델 비교 실험을 통해 특정 모델의 과적합 가능성도 사전 검토함

● 재사용성

- 저장된 모델은 .pt 포맷과 Hugging Face 포맷 둘 다 가능하여
  다양한 추론 환경(로컬, 클라우드, REST API 등)에서 손쉽게 재사용 가능

- Tokenizer 및 Config까지 함께 저장되어 모델 로딩과 추론 구성 용이

● 향후 활용 방안

- 법률 챗봇 응답 조건 분기 로직에서 분류 결과(인용/기각 등)를 기반으로 직접 사용 가능

- 유사 판례 검색 시스템의 전처리 필터로 활용하여,
  특정 분류 결과에 따라 검색 결과 정렬 또는 요약 방식 제어 가능

- 법률 도메인 LLM 학습데이터 구축 시,
  실제 판례에 대해 자동 라벨링을 제공하는 데이터 증강 도구로 활용 가능

- 학습된 모델은 ONNX 변환 또는 FastAPI 배포를 통해 실시간 판례 분류 서비스로도 확장 가능